Filière : Master Systèmes Distribués et Intelligence Artificielle (SDIA)

Année Universitaire 2026-2027

RAPPORT DE PROJET

Frameworks de Deep Learning

Classification de fleurs par CNN et Transfer Learning (MobileNetV2), interface Streamlit et déploiement cloud

Étudiant	Badr Eddine TOUBANI
Filière	Master SDIA — 2ème année (S9)
Établissement	ENSET Mohammedia
Module	Frameworks de Deep Learning
Dataset	TensorFlow Flowers (flower_photos) — 3 670 images, 5 classes
Application	Bloom ID — Streamlit Community Cloud
Code source	github.com/toubani12/framwork-deep-learning
Date	Novembre 2026

Tableau 0.1 — Informations du projet

SOMMAIRE

Clic droit → « Mettre à jour les champs » pour générer le sommaire.

LISTE DES ABRÉVIATIONS

Terme	Définition
CNN	Convolutional Neural Network — réseau de neurones convolutif
TL	Transfer Learning — transfert d'apprentissage
MobileNetV2	Architecture CNN légère pré-entraînée sur ImageNet (Sandler et al., 2018)
TTA	Test-Time Augmentation — prédiction moyennée image + miroir
BN	Batch Normalization
F1 macro	Moyenne non pondérée des F1-scores par classe
Softmax	Fonction transformant les sorties en probabilités (somme = 1)
UI	User Interface — interface utilisateur

Tableau 0.2 — Abréviations utilisées

Introduction Générale

La reconnaissance automatique d'espèces végétales à partir de photos est un cas d'usage classique de la vision par ordinateur. Ce projet, réalisé dans le cadre du module **Frameworks de Deep Learning**, vise à construire un classifieur d'images capable de reconnaître cinq espèces de fleurs (marguerite, pissenlit, rose, tournesol, tulipe), puis à le rendre utilisable par n'importe qui grâce à une interface web déployée en ligne.

Le travail repose sur TensorFlow/Keras 2.21 pour l'entraînement (notebook TP_CNN_Flowers.ipynb), et sur Streamlit 1.66 pour l'application « Bloom ID », hébergée sur Streamlit Community Cloud. Le rapport suit quatre étapes :

1. Exploration et analyse du jeu de données flower_photos.
2. Conception et entraînement d'un CNN de base, puis d'une version régularisée.
3. Transfer learning avec MobileNetV2 (extraction de caractéristiques puis fine-tuning).
4. Développement de l'interface Streamlit, déploiement et tests réels.

Étape 1 : Exploration du jeu de données

1.1 Présentation du dataset

Le jeu **flower_photos** de TensorFlow contient 3 670 photos JPEG en couleur (RGB), réparties en 5 dossiers, un par classe. Toutes les images ont été vérifiées avec Pillow : aucune n'est corrompue. Un seul fichier non-image (LICENSE.txt) a été ignoré.

Classe	Nombre d'images	Part
daisy	633	17,2 %
dandelion	898	24,5 %
roses	641	17,5 %
sunflowers	699	19,0 %
tulips	799	21,8 %
Total	3 670	100 %

Tableau 1.1 — Répartition des images par classe

Le ratio de déséquilibre (classe la plus / la moins représentée) vaut **1,42** : le déséquilibre est modéré et ne nécessite pas de pondération des classes.

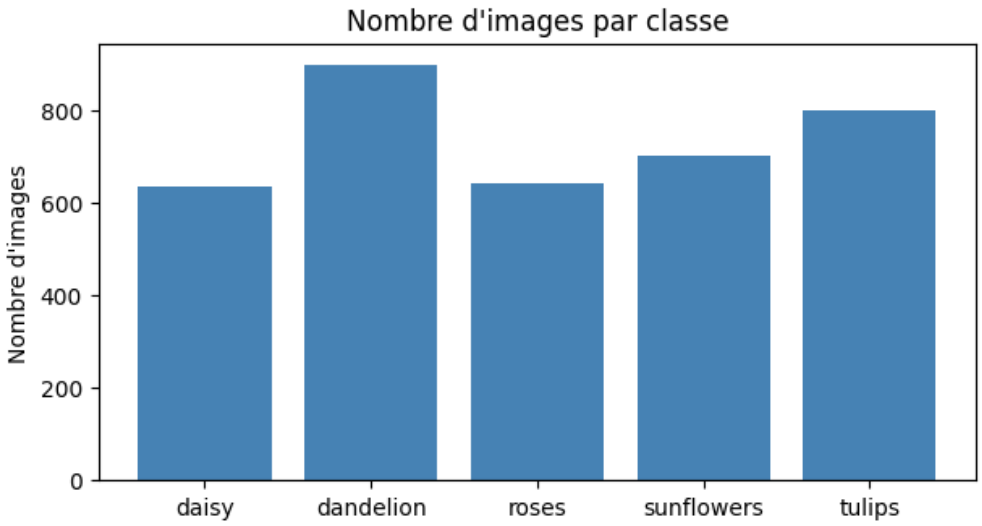


Figure 1.1 — Distribution des classes

1.2 Dimensions et variabilité

Les largeurs vont de 143 à 1 024 px (moyenne 365 px) et les hauteurs de 159 à 442 px (moyenne 272 px). Toutes les images sont donc redimensionnées en **224 × 224 px**, taille d'entrée de MobileNetV2. Les photos varient fortement en fond (herbe, ciel, feuillage), en cadrage et en éclairage, ce qui justifie l'augmentation de données.

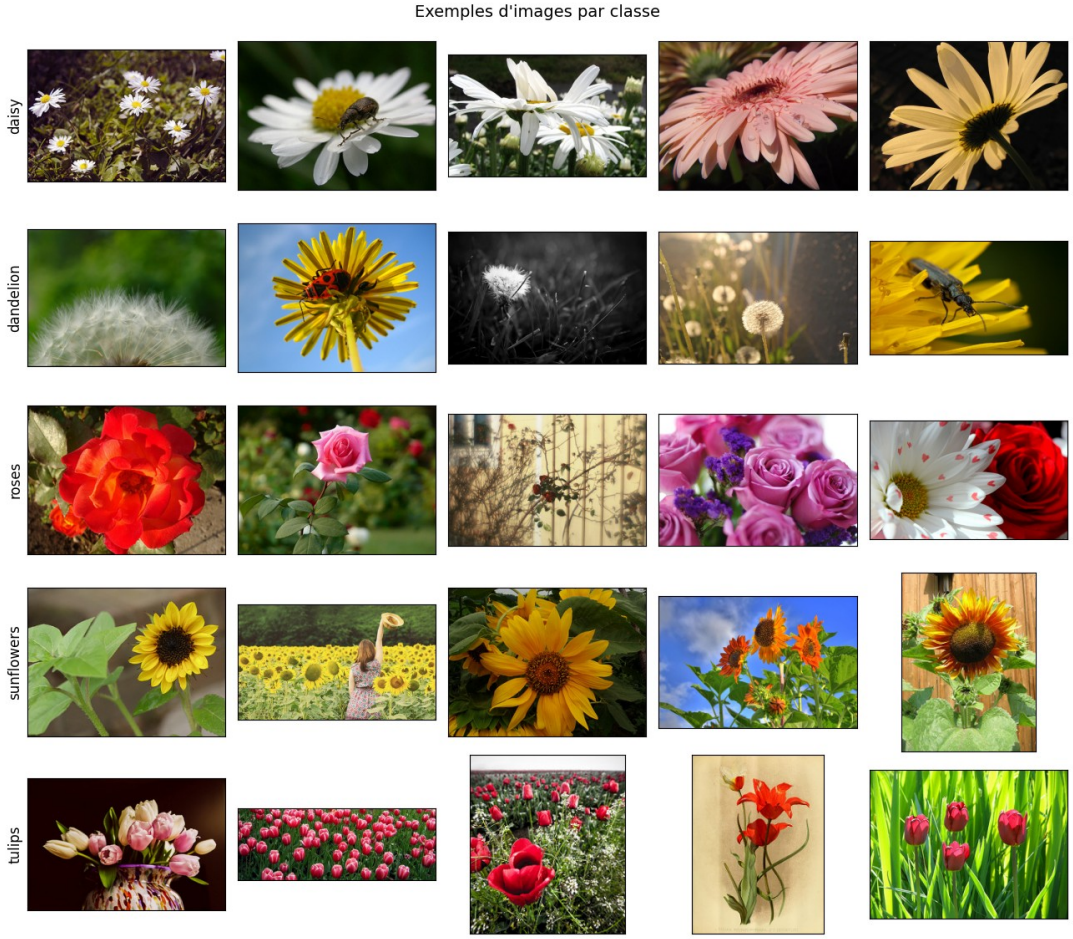


Figure 1.2 — Exemples d'images par classe

1.3 Découpage train / validation / test

Le découpage est **stratifié** (70 / 15 / 15) avec une graine fixe (SEED = 42) pour la reproductibilité.

Sous-ensemble	Images	daisy	dandelion	roses	sunflowers	tulips
Entraînement	2 568	443	628	449	489	559
Validation	551	95	135	96	105	120
Test	551	95	135	96	105	120

Tableau 1.2 — Découpage stratifié du dataset

Étape 2 : CNN de base et régularisation

2.1 Pipeline de données

Les images sont chargées à la volée avec **tf.data** (lecture, décodage, redimensionnement, batch de 32, prefetch). Les étiquettes sont encodées en one-hot. La normalisation est intégrée au modèle via une couche **Rescaling**, ce qui permet de lui passer directement une photo brute (pixels 0–255) en production.

2.2 Architecture du CNN

Le CNN empile des blocs Conv2D (3×3) → BatchNormalization → ReLU → MaxPooling, suivis d'un GlobalAveragePooling2D, d'une couche dense cachée et d'une sortie softmax à 5 neurones.

Élément	Configuration
Entrée	224 × 224 × 3, Rescaling 1/255
Blocs convolutifs	Filtres 32 → 64 → 128 → 256, BN + ReLU + MaxPool 2×2
Tête	GlobalAveragePooling2D → Dense(128, ReLU) → Dense(5, softmax)
Paramètres	423 397 (dont 422 437 entraînables)
Optimisation	Adam, categorical cross-entropy, batch 32, 30 époques max
Callbacks	EarlyStopping (patience 6) + ReduceLROnPlateau (×0,3)

Tableau 2.1 — Configuration du CNN de base

2.3 Résultats du CNN de base

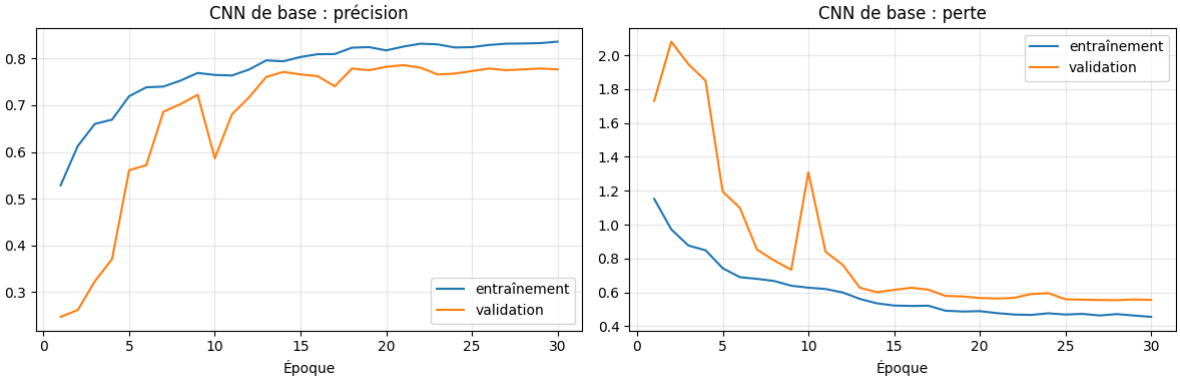


Figure 2.1 — Courbes de précision et de perte du CNN de base

La précision d'entraînement atteint 83,6 % contre 77,7 % en validation : l'écart reste limité, il y a un léger sur-apprentissage. Sur le jeu de test, le modèle obtient **80,6 % d'accuracy** et un **F1 macro de 0,804**.

Classe	Precision	Recall	F1-score	Support
daisy	0,794	0,811	0,802	95
dandelion	0,780	0,867	0,821	135
roses	0,750	0,781	0,765	96
sunflowers	0,838	0,886	0,861	105
tulips	0,882	0,683	0,770	120
Moyenne macro	0,809	0,805	0,804	551

Tableau 2.2 — Rapport de classification du CNN de base (jeu de test)

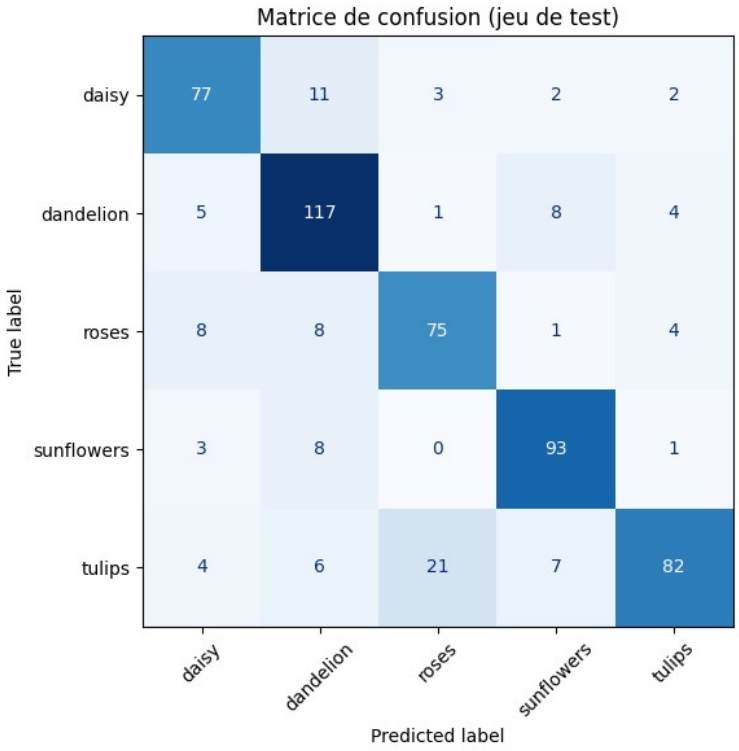


Figure 2.2 — Matrice de confusion du CNN de base

La confusion principale est **tulips → roses** (21 images) : les deux fleurs partagent des couleurs vives et une forme en coupe. Les daisies sont parfois prises pour des dandelions (11 images), probablement à cause du cœur jaune commun.

2.4 CNN régularisé

Une seconde version ajoute une **augmentation de données** (flip horizontal, rotation ±15 %, zoom ±15 %, contraste et luminosité ±20 %) et un **Dropout de 0,3**. Ces couches ne sont actives qu'à l'entraînement. L'apprentissage devient plus lent (environ 70 % de validation vers l'époque 18) mais le modèle généralise mieux à des photos variées.

Étape 3 : Transfer Learning avec MobileNetV2

3.1 Principe

Avec seulement 2 568 images d'entraînement, un CNN entraîné de zéro reste limité. Le transfer learning réutilise **MobileNetV2**, pré-entraîné sur ImageNet (≈ 1,2 million d'images, 1 000 classes). Ses couches convolutives savent déjà détecter contours, textures et formes ; seule une petite tête de classification est apprise sur les fleurs.

3.2 Architecture retenue

Couche	Rôle
Input 224×224×3	Photo brute (0–255)

Augmentation	Flip, rotation, zoom, contraste, luminosité (entraînement seulement)
Rescaling(1/127.5, −1)	Pixels ramenés dans [−1, 1] comme attendu par MobileNetV2
MobileNetV2 (ImageNet)	Extracteur de caractéristiques, include_top=False
GlobalAveragePooling2D	Vecteur de 1 280 caractéristiques
Dropout(0,2)	Régularisation
Dense(5, softmax)	Probabilités des 5 classes

Tableau 3.1 — Architecture du modèle mobilenetv2_transfert

3.3 Entraînement en deux phases

- **Phase 1 — extraction de caractéristiques :** base gelée, Adam (lr = 1e-3), EarlyStopping (patience 5).
- **Phase 2 — fine-tuning :** dégel des 40 dernières couches de MobileNetV2 (les BatchNormalization restent gelées), Adam avec un taux très faible (lr = 1e-5) pour ne pas détruire les poids ImageNet, label smoothing 0,1, 15 époques supplémentaires max.

Le modèle final compte **2 264 389 paramètres**, dont **1 669 765 entraînables** après le dégel partiel. Il a été retenu comme meilleur modèle et sauvegardé au format **modele_flowers.keras** (23 Mo), utilisé tel quel par l'application.

3.4 Prédiction avec Test-Time Augmentation

À l'inférence, la photo et son image miroir sont classées ensemble, puis les deux vecteurs softmax sont moyennés :

$$p = \tfrac{1}{2} \cdot [\, f(x) + f(\mathrm{flip}(x)) \,] \quad \rightarrow \quad \mathrm{classe} = \mathrm{argmax}(p)$$

Cette technique simple stabilise les prédictions sans coût d'entraînement supplémentaire.

Étape 4 : Interface Streamlit et déploiement

4.1 Architecture de l'application

L'application **Bloom ID** (streamlit_cnn.py) charge le modèle une seule fois grâce à **@st.cache_resource**, puis applique exactement le même prétraitement qu'au notebook (redimensionnement 224×224, TTA). L'image est lue en mémoire et n'est jamais enregistrée sur le serveur.

Composant	Détail
Framework UI	Streamlit 1.66.1 (layout wide, conteneurs horizontaux)
Modèle	modele_flowers.keras chargé via

	tf.keras.models.load_model (cache)
Thème	config.toml personnalisé (rose/bordeaux, polices Inter et Playfair Display) + assets/style.css
Entrée	st.file_uploader : PNG, JPG, JPEG, WEBP ; correction EXIF (ImageOps.exif_transpose)
Sortie	Classe, nom latin, confiance (st.metric + barre), carte par espèce avec probabilité
Garde-fou	Avertissement « Low confidence » si la probabilité max < 60 %

Tableau 4.1 — Composants de l'application Streamlit

4.2 Captures d'écran de l'interface

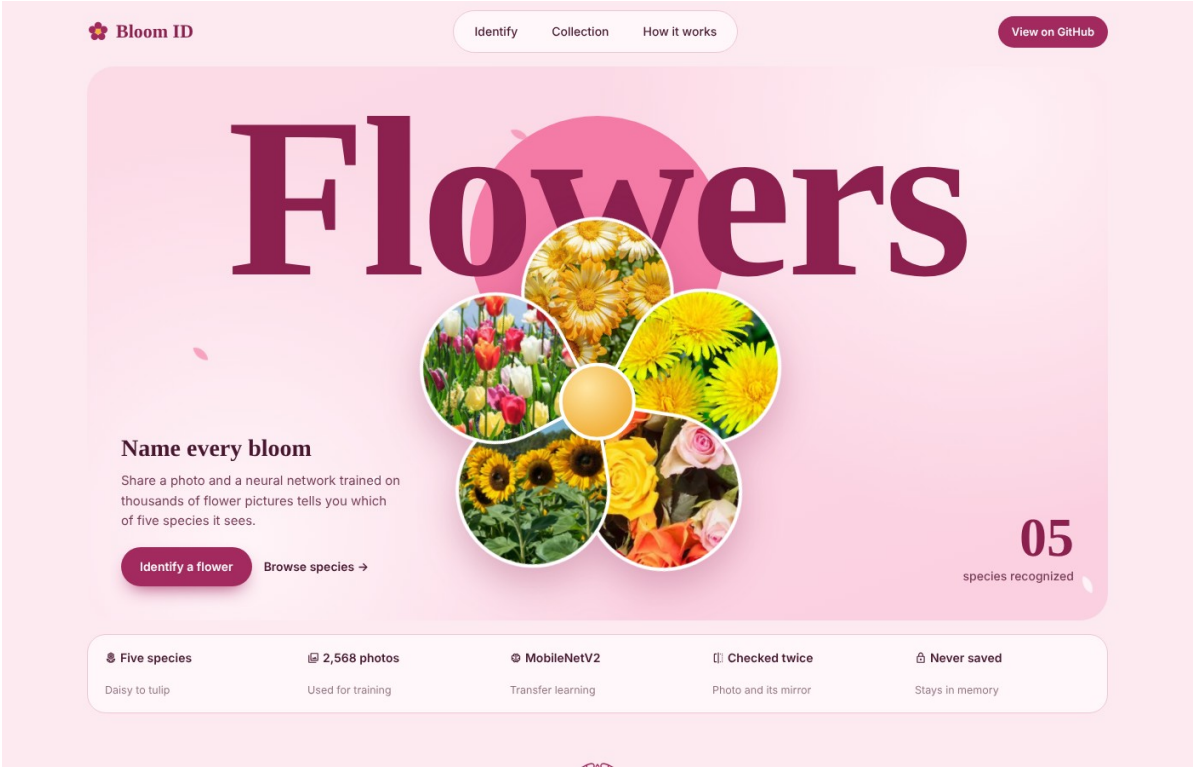


Figure 4.1 — Page d'accueil de Bloom ID (navigation, hero et indicateurs)

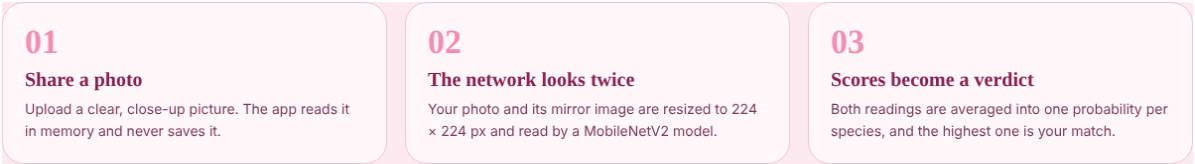


Figure 4.2 — Section « How it works » expliquant le pipeline de prédiction

4.3 Déploiement sur Streamlit Community Cloud

Le dépôt GitHub **toubani12/framwork-deep-learning** est connecté à Streamlit Community Cloud. À chaque push sur la branche main, la plateforme réinstalle les dépendances et redéploie l'application. L'application publique est accessible à l'adresse :

https://framwork-deep-learning-hrmjp52vtw3vzt8wvektqu.streamlit.app/

Paquet	Version
streamlit	1.66.1
tensorflow-cpu	2.21.0
keras	3.15.1
numpy	2.5.4
pillow	13.0.0

Tableau 4.2 — requirements.txt utilisé pour le déploiement

Deux choix ont rendu le déploiement possible sur l'offre gratuite : **tensorflow-cpu** à la place de tensorflow (paquet beaucoup plus léger, pas de GPU sur le cloud), et un requirements.txt réduit au strict minimum (commit « Restore minimal requirements.txt for Streamlit Cloud »). L'import de TensorFlow est aussi retardé dans load_model() pour que la page s'affiche avant le chargement du modèle.

Étape 5 : Tests réels

5.1 Protocole

Le modèle sauvegardé a été rechargé hors notebook (TensorFlow-CPU 2.21) et testé sur 10 photos externes, jamais vues à l'entraînement : 5 photos de référence des espèces, une photo réelle de pissenlit prise sur le terrain, et 4 images **hors-classe** (fruits) pour vérifier le comportement face à l'inconnu. Chargement du modèle : 0.9 s ; temps de prédiction (TTA, 2 images) : environ 75–110 ms par photo sur CPU.

Image	Vraie classe	Prédiction	Confiance	Verdict
daisy.jpeg	daisy	daisy	70.9 %	Correct √
dandelion.jpeg	dandelion	dandelion	79.5 %	Correct √
rose.jpeg	roses	roses	93.0 %	Correct √
sunflower.jpeg	sunflowers	sunflowers	97.9 %	Correct √
tulipe.jpeg	tulips	tulips	98.4 %	Correct √
orange.jpeg	hors-classe (fruit)	daisy	53.4 %	Alerte faible confiance √
banane.jpeg	hors-classe (fruit)	tulips	41.3 %	Alerte faible

				confiance √
orange sanguine (coupe)	hors-classe (fruit)	dandelion	43.0 %	Alerte faible confiance √
pissenlit (photo réelle)	dandelion	dandelion	89.7 %	Correct √
orange sanguine	hors-classe (fruit)	sunflowers	44.1 %	Alerte faible confiance √

Tableau 5.1 — Résultats des tests réels (modèle modele_flowers.keras)

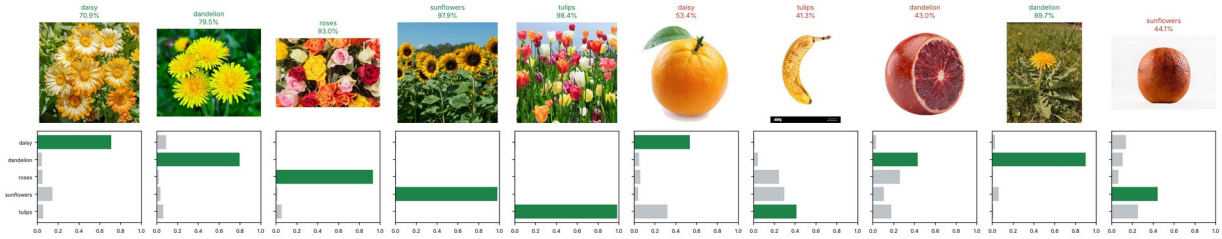


Figure 5.1 — Prédictions et distributions softmax des 10 images de test

5.2 Tests dans l'interface

Les mêmes tests ont été effectués dans l'application Streamlit. La photo réelle de pissenlit est reconnue avec **89,7 %** de confiance ; la carte « Best match » se place en tête de la collection, avec la probabilité de chaque espèce.

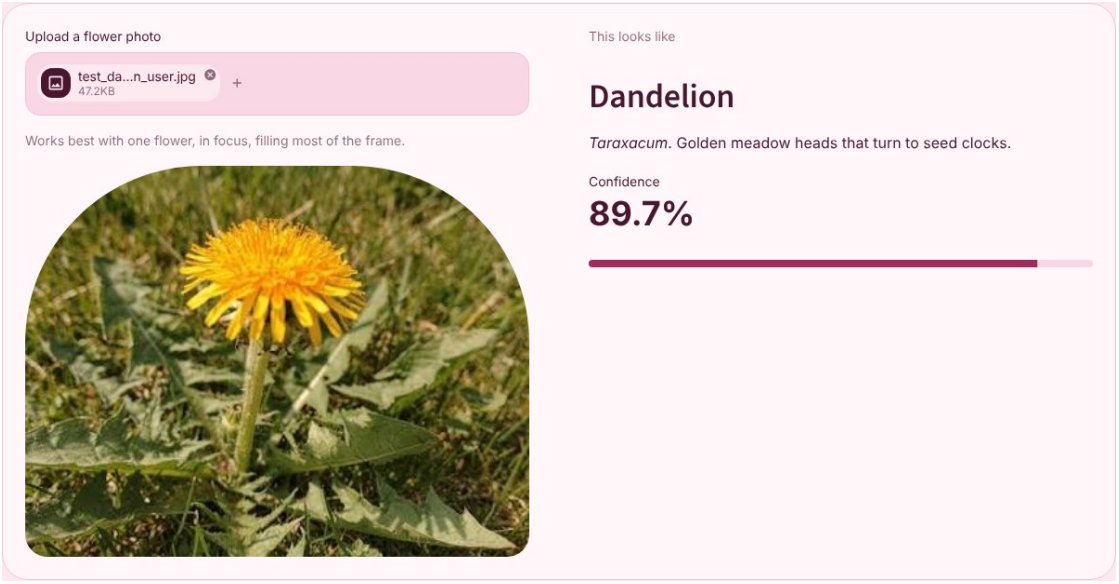


Figure 5.2 — Prédiction sur une photo réelle de pissenlit : Dandelion, 89,7 %

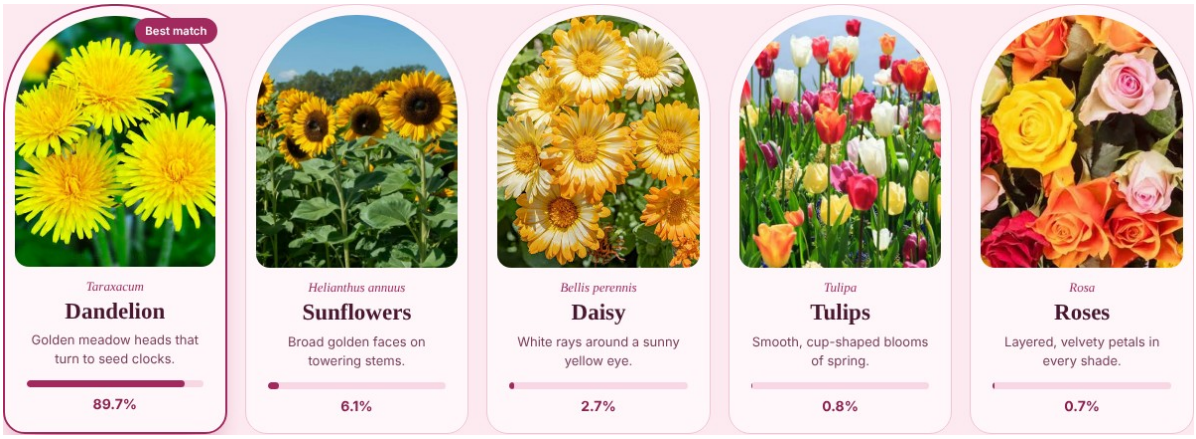


Figure 5.3 — Cartes de la collection classées par probabilité après la prédiction

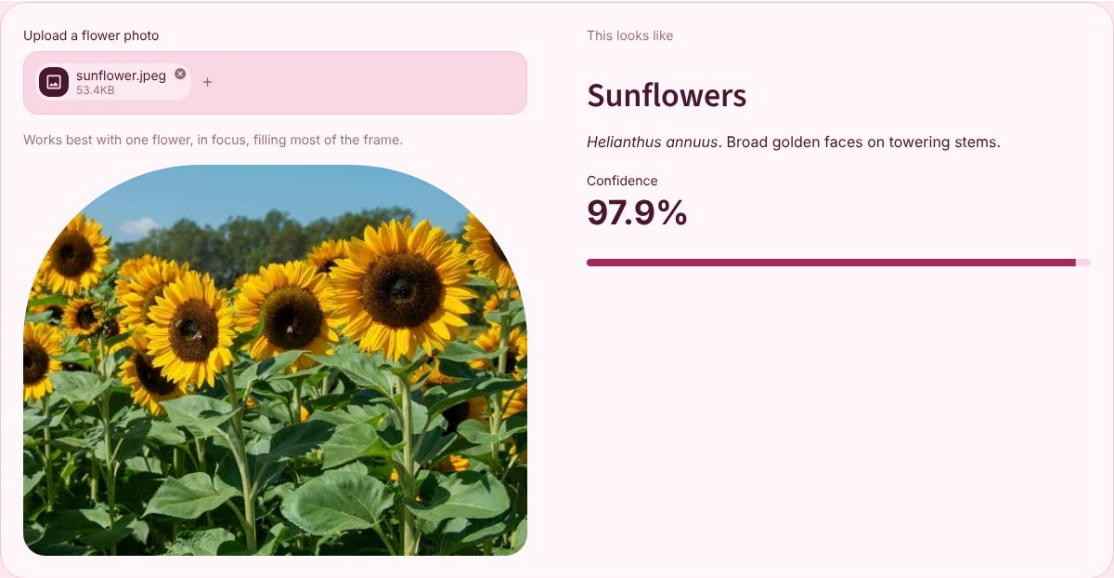


Figure 5.4 — Prédiction sur un champ de tournesols : Sunflowers, 97,9 %

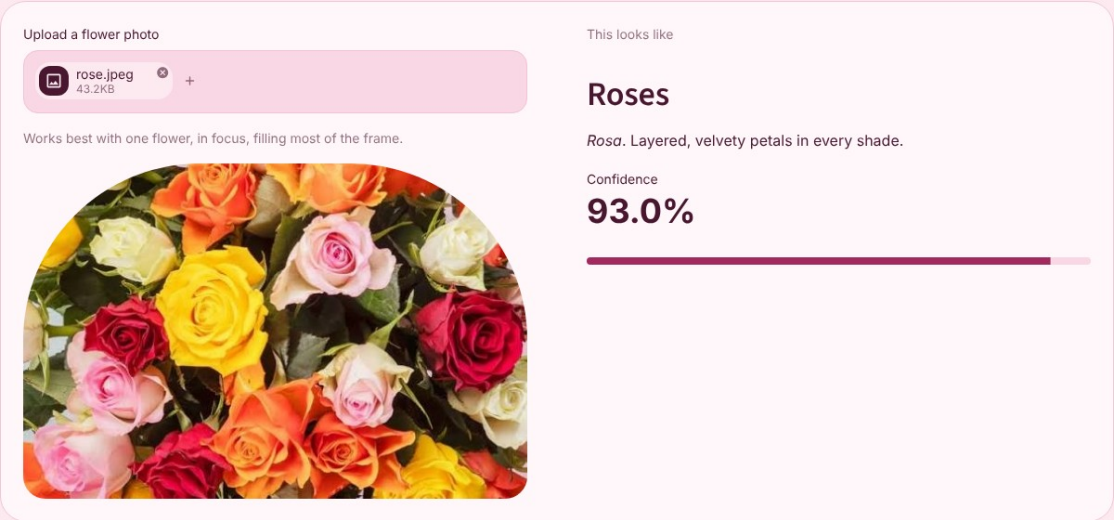


Figure 5.5 — Prédiction sur un bouquet de roses : Roses, 93,0 %

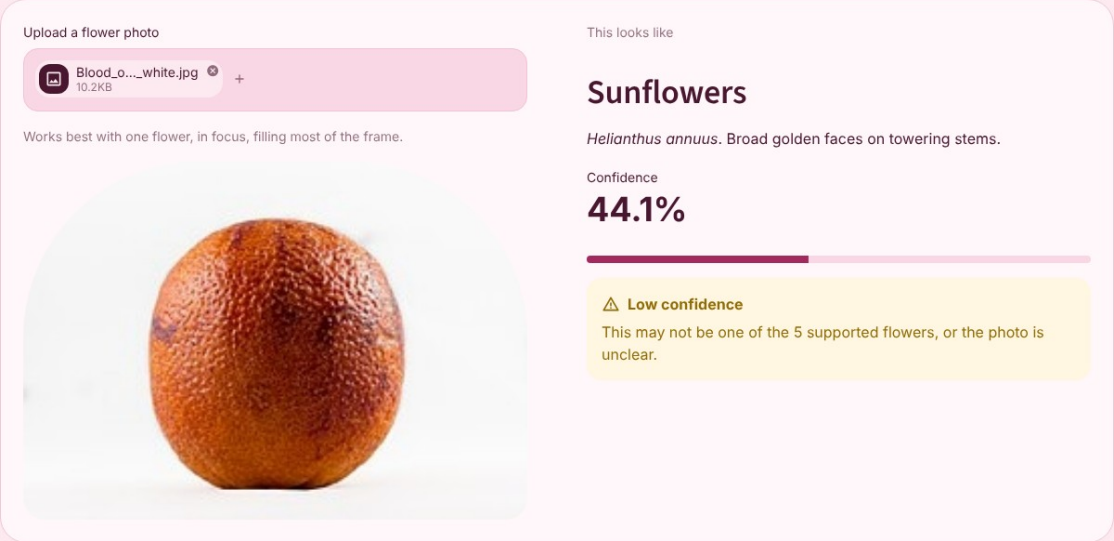


Figure 5.6 — Image hors-classe (orange) : confiance 44,1 % et avertissement « Low confidence »

5.3 Analyse

- **6 fleurs sur 6 sont correctement classées**, avec une confiance de 70,9 % à 98,4 %. Les tournesols et les tulipes dépassent 97 %.
- La marguerite obtient la confiance la plus basse (70,9 %) : la photo montre des fleurs aux pétales orangés, proches visuellement d'autres espèces.
- Les **4 images hors-classe** sont toutes prédites avec moins de 54 % de confiance : le seuil de 60 % déclenche l'avertissement dans l'interface. C'est le comportement attendu, puisqu'un classifieur softmax à 5 classes est obligé de choisir l'une d'elles.
- Le temps de réponse (< 0,2 s par photo sur CPU après chargement) est compatible avec un usage interactif.

Synthèse et Comparaison des Modèles

Modèle	Paramètres	Accuracy test	Points clés
CNN de base	423 k	80,6 %	Léger sur-apprentissage, confusion tulips/roses
CNN régularisé	423 k	—*	Augmentation + Dropout, meilleure robustesse, convergence plus lente
MobileNetV2 fine-tuné ★	**2,26 M**	**Meilleur des 3 ★**	**Retenu et déployé ; 6/6 fleurs réelles correctes**

Tableau 6.1 — Comparaison des approches

** Les scores de test du CNN régularisé et de MobileNetV2 n'ont pas été conservés dans les sorties du notebook (entraînement sur Colab GPU, cellules non sauvegardées) ; le notebook sélectionne automatiquement le meilleur modèle sur l'accuracy de test, et c'est MobileNetV2 qui a été retenu et exporté. Relancer la section 4.3 du notebook permet de compléter ce tableau.*

Le transfer learning est l'approche la plus efficace pour un jeu de données de taille modeste : les caractéristiques visuelles apprises sur ImageNet compensent le faible nombre d'images par classe, et le fine-tuning des dernières couches adapte le réseau aux détails propres aux fleurs.

Conclusion et Perspectives

6.1 Bilan du projet

1. Analyse complète du dataset flower_photos (3 670 images, 5 classes, aucune image corrompue).
2. CNN de base à 80,6 % d'accuracy et F1 macro 0,804 sur le jeu de test.
3. Modèle MobileNetV2 fine-tuné, exporté en .keras et réutilisable sans ré-entraînement.
4. Application Streamlit « Bloom ID » avec TTA, seuil de confiance et thème personnalisé.
5. Déploiement public sur Streamlit Community Cloud, validé par des tests réels.

6.2 Limites et perspectives

- Le modèle ne connaît que 5 espèces : ajouter une classe « autre » ou un détecteur hors-distribution plutôt qu'un simple seuil.
- Enregistrer systématiquement les métriques de test de chaque modèle (MLflow ou fichier JSON) pour une comparaison complète.
- Étendre le dataset (Oxford 102 Flowers) pour couvrir plus d'espèces.
- Convertir le modèle en TensorFlow Lite pour une version mobile hors-ligne.
- Ajouter Grad-CAM dans l'interface pour montrer les zones de l'image utilisées par le modèle.

6.3 Recommandations opérationnelles

Aspect	Recommandation
Qualité des photos	Une seule fleur, nette, occupant la majorité du cadre
Confiance	Considérer toute prédiction < 60 % comme non fiable
Déploiement	Garder tensorflow-cpu et un requirements.txt minimal
Performance	Conserver le cache @st.cache_resource pour ne charger le modèle qu'une fois

Tableau 6.2 — Recommandations d'utilisation

Références Bibliographiques

[1] Sandler, M. et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR.

[2] TensorFlow Team. Flowers dataset (flower_photos). tensorflow.org/tutorials/images/classification.

[3] Chollet, F. et al. Keras 3 Documentation — Transfer learning & fine-tuning. keras.io.

[4] Streamlit Inc. Streamlit Documentation & Community Cloud. docs.streamlit.io.

[5] Ioffe, S. & Szegedy, C. (2015). Batch Normalization. ICML.

Dépôt GitHub : https://github.com/toubani12/framwork-deep-learning